

Day 1 종합실습 결과보고서

과목명	데이터분석 및 AIOps — 1. 데이터 분석을 위한 Python 이해
과제명	Day 1 종합실습 — 데이터 수집 미니 파이프라인
캠퍼스_반	판교_3반
고유번호_이름	P102_황도희
제출일	2026-08-03

개요

목적

외부 **API 3종**을 대상으로 한 데이터 수집·검증·저장 파이프라인을 구현하여, 비동기 처리와 스키마 검증, 파일 포맷 성능 비교, 자동화 테스트까지 이어지는 데이터 엔지니어링 기본 워크플로우를 실습한다.

범위

- 데이터 소스: **Open-Meteo**(날씨), **Countries.dev**(국가정보), **ip-api**(IP 위치정보)
- 처리 단계: 비동기 수집 → 스키마 검증(**Pydantic v2**) → 이중 포맷 저장(**CSV/Parquet**) → 성능 측정 → 자동화 테스트

요약

asyncio.gather() 기반으로 3개 API를 동시 호출하는 구조로 설계하여 순차 호출 대비 총 소요 시간을 최소화했다. 수집된 데이터는 **Pydantic v2** 모델로 타입 및 값 범위(기온, 확률, 위경도 등)를 검증했으며, 검증을 통과한 데이터만 **CSV·Parquet** 두 포맷으로 저장해 형식별 처리 성능을 비교했다. 전체 로직에 대해 **pytest** 기반 자동화 테스트 9건을 작성하여 정상/이상 케이스를 모두 검증했고, **ruff** 정적 분석 결과 이슈 없음을 확인했다. 산출물은 **Git**으로 버전 관리하고 원격 저장소에 반영을 완료했다.

실행

아래는 구현한 파이프라인의 실행 과정을 단계별로 캡처한 화면이다.

가상환경 활성화

```
● → day1종합실습 git:(main) source venv/bin/activate
● (venv) → day1종합실습 git:( source /Users/hdh22/.../4.../0803.../day1종합실습/venv/bin/activate
○ (venv) → day1종합실습 git:(main) □
```

venv를 생성하고 활성화한 뒤, 이후 모든 패키지 설치 및 실행이 이 가상환경 내에서 이루어지도록 구성했다.

requirements.txt

```

● (venv) → day1종합실습 git:(main) cat requirements.txt
annotated-types==0.8.0
anyio==4.14.2
certifi==2026.7.22
h11==0.16.0
httpcore==1.0.9
httpx==0.28.1
idna==3.18
iniconfig==2.3.0
numpy==2.4.6
packaging==26.2
pandas==3.0.5
pluggy==1.6.0
pyarrow==25.0.0
pydantic==2.13.4
pydantic_core==2.46.4
Pygments==2.20.0
pytest==9.1.1
python-dateutil==2.9.0.post0
ruff==0.16.1
six==1.17.0
typing-inspection==0.4.2
typing_extensions==4.16.0
○ (venv) → day1종합실습 git:(main) █

```

httpx, pydantic, pandas, pyarrow, pytest, ruff 등 필요한 패키지 목록을 `pip freeze`로 고정하여 관리했다.

파이프라인 전체 실행 결과

```

● (venv) → day1종합실습 git:(main) python3 src/pipeline.py
[응답 확인] country: status=200
[응답 확인] ip: status=200
[응답 확인] weather: status=200
[검증] weather 레코드 수 : 72
[검증] weather 샘플 : time='2026-08-03T00:00' temperature=25.3 precipitation_probability=0
[검증] country: name='Korea (Republic of)' capital='Seoul' population=51780579
[검증] ip: country='United States' city='Ashburn' lat=39.03 lon=-77.5
[성능] weather CSV - 쓰기 : 0.004101s / 읽기 : 0.003537s
[성능] weather Parquet - 쓰기 : 0.035534s / 읽기 : 0.032278s
[저장] country -> country_output.csv / country_output.parquet
[저장] ip -> ip_output.csv / ip_output.parquet
○ (venv) → day1종합실습 git:(main) █

```

3개 API에 대한 응답 상태코드 확인, Pydantic 검증 결과(weather 72건, country/ip 각 1건), CSV/Parquet 저장 및 성능 측정 결과가 순서대로 출력되었다.

생성된 파일 목록

```

D (venv) → day1종합실습 git:(main) ls -al
total 80
drwxr-xr-x@ 18 hdh22 staff 576 Aug 3 16:38 .
drwxr-xr-x  9 hdh22 staff 288 Aug 3 16:04 ..
drwxr-xr-x@ 14 hdh22 staff 448 Aug 3 18:20 .git
-rw-r--r--@  1 hdh22 staff  39 Aug 3 16:14 .gitignore
drwxr-xr-x@  6 hdh22 staff 192 Aug 3 16:21 .pytest_cache
drwxr-xr-x@  5 hdh22 staff 160 Aug 3 16:21 .ruff_cache
-rw-r--r--@  1 hdh22 staff  42 Aug 3 16:26 README.md
-rw-r--r--@  1 hdh22 staff  59 Aug 3 18:26 country_output.csv
-rw-r--r--@  1 hdh22 staff 2707 Aug 3 18:26 country_output.parquet
-rw-r--r--@  1 hdh22 staff  55 Aug 3 18:26 ip_output.csv
-rw-r--r--@  1 hdh22 staff 3153 Aug 3 18:26 ip_output.parquet
-rw-r--r--@  1 hdh22 staff 1788 Aug 3 18:26 output.csv
-rw-r--r--@  1 hdh22 staff 3801 Aug 3 18:26 output.parquet
-rw-r--r--@  1 hdh22 staff  374 Aug 3 16:13 requirements.txt
drwxr-xr-x@  4 hdh22 staff 128 Aug 3 16:21 src
drwxr-xr-x@  4 hdh22 staff 128 Aug 3 16:21 tests
-rw-r--r--@  1 hdh22 staff 213 Aug 3 16:34 validated_summary.json
drwxr-xr-x@  6 hdh22 staff 192 Aug 3 16:08 venv
D (venv) → day1종합실습 git:(main) █

```

검증을 통과한 weather, country, ip 데이터가 각각 CSV·Parquet 두 형식으로 정상 저장되었음을 확인했다.

pytest 결과

```

D (venv) → day1종합실습 git:(main) ls -al
total 80
drwxr-xr-x@ 18 hdh22 staff 576 Aug 3 16:38 .
drwxr-xr-x  9 hdh22 staff 288 Aug 3 16:04 ..
drwxr-xr-x@ 14 hdh22 staff 448 Aug 3 18:20 .git
-rw-r--r--@  1 hdh22 staff  39 Aug 3 16:14 .gitignore
drwxr-xr-x@  6 hdh22 staff 192 Aug 3 16:21 .pytest_cache
drwxr-xr-x@  5 hdh22 staff 160 Aug 3 16:21 .ruff_cache
-rw-r--r--@  1 hdh22 staff  42 Aug 3 16:26 README.md
-rw-r--r--@  1 hdh22 staff  59 Aug 3 18:26 country_output.csv
-rw-r--r--@  1 hdh22 staff 2707 Aug 3 18:26 country_output.parquet
-rw-r--r--@  1 hdh22 staff  55 Aug 3 18:26 ip_output.csv
-rw-r--r--@  1 hdh22 staff 3153 Aug 3 18:26 ip_output.parquet
-rw-r--r--@  1 hdh22 staff 1788 Aug 3 18:26 output.csv
-rw-r--r--@  1 hdh22 staff 3801 Aug 3 18:26 output.parquet
-rw-r--r--@  1 hdh22 staff  374 Aug 3 16:13 requirements.txt
drwxr-xr-x@  4 hdh22 staff 128 Aug 3 16:21 src
drwxr-xr-x@  4 hdh22 staff 128 Aug 3 16:21 tests
-rw-r--r--@  1 hdh22 staff 213 Aug 3 16:34 validated_summary.json
drwxr-xr-x@  6 hdh22 staff 192 Aug 3 16:08 venv
D (venv) → day1종합실습 git:(main) █

```

정상 케이스(3건), 범위 오류 케이스(4건), 타입 오류 케이스(1건), 필드 누락 케이스(1건)를 포함한 총 9건의 테스트가 모두 통과했다.

ruff 결과

```

D (venv) → day1종합실습 git:(main) python3 -m ruff check .
All checks passed!
D (venv) → day1종합실습 git:(main) █

```

정적 분석 결과 스타일/린트 이슈가 발견되지 않았다.

git 커밋 이력

```
...skipping...
6f48b1a (HEAD -> main, origin/main, origin/HEAD) 오류 테스트 케이스 추가, (주석 최종 작업 전)
078c8ce Pydantic 범위 검증 (Field ge/le) 추가, country/ip 저장, 검증 실패 테스트 케이스 추가
d767d67 Merge branch 'main' of https://github.com/ahrdyrxkddhfl/SKALA-project
b5b8e97 Day1 종합실습: 데이터 수집 파이프라인 구현
a44bac9 Initial commit
```

기능 구현부터 검증 로직 보강까지의 작업 이력이 커밋 단위로 기록되어 있으며, 원격 저장소(GitHub)에 반영이 완료되었다.

Code 분석 결과에 대해

1. 구현 개요

항목	내용
비동기 처리	<code>httpx.AsyncClient + asyncio.gather(return_exceptions=True)</code> 로 3개 API 동시 호출, 개별 요청 실패가 전체 파이프라인에 영향을 주지 않도록 격리 처리
스키마 검증	Pydantic v2 <code>BaseModel + Field(ge/le/gt)</code> 로 타입 검증과 값 범위 검증을 동시에 적용
예외 처리	<code>raw.get()</code> 기반 필드 접근으로 <code>KeyError</code> 를 <code>ValidationError</code> 로 일원화, 단일 예외 처리 경로로 단순화
저장	검증 통과 데이터를 CSV/Parquet 이중 포맷으로 저장, 쓰기/읽기 소요 시간 각각 측정
테스트	정상 케이스 3건, 범위 오류 4건, 타입 오류 1건, 필드 누락 1건 — 총 9건의 <code>pytest</code> 케이스로 검증 로직의 정확성을 확인

2. 주요 설계 결정 및 근거

2.1 동시성 확보: 3개 API를 순차 호출하지 않고 `asyncio.gather()`로 동시 처리한 이유는, I/O 대기 시간이 지배적인 네트워크 호출 특성상 병렬화가 전체 처리 시간 단축에 직접적으로 기여하기 때문이다. `return_exceptions=True` 옵션을 적용해, 특정 API의 장애가 전체 파이프라인의 중단으로 이어지지 않도록 설계했다.

2.2 검증 범위 확장: 요구사항이 "타입·범위 검증"으로 명시되어 있어, 단순 타입 체크에 그치지 않고 `Field(ge=, le=)`를 활용해 논리적으로 불가능한 값(예: 강수확률 150%, 위도 999도)까지 걸러지도록 구현했다. 이는 실제 운영 환경에서 API 스펙 변경이나 응답 이상치로 인한 데이터 오염을 방지하는 것과 동일한 목적을 가진다.

2.3 성능 측정 대상 선정: 성능 비교는 표본 수가 충분한 **weather** 데이터(72건)를 기준으로 수행했다. **country/ip**는 각 1건에 불과해 포맷 간 처리 시간 차이가 통계적으로 의미를 갖기 어렵다고 판단, 저장은 두 포맷 모두 수행하되 성능 측정의 대표 지표로는 포함하지 않았다.

3. 검증 결과 요약

- **weather 72건, country/ip 각 1건** — 전체 데이터 **Pydantic** 검증 통과
- 소규모 데이터(72건) 환경에서 **CSV**가 **Parquet** 대비 쓰기/읽기 모두 더 빠른 결과를 보임(각각 약 **0.004s vs 0.035s**). 이는 **Parquet**의 컬럼 지향 저장 방식과 **pyarrow** 엔진 초기화 비용이 소규모 데이터에서는 오히려 오버헤드로 작용하기 때문으로 판단되며, 대용량 데이터 환경에서는 결과가 역전될 가능성이 높다.
- **pytest 9건** 전체 통과, **ruff** 정적 분석 이슈 **0건**

4. 개선 제안

- 재시도 로직 부재: 현재 네트워크 요청 실패 시 재시도 없이 즉시 실패 처리된다. 운영 환경 적용 시 **tenacity** 등을 활용한 지수 백오프 재시도 로직 추가가 필요하다.
 - 설정 외부화: **API** 엔드포인트가 코드 내 딕셔너리로 하드코딩되어 있어, **API** 추가/변경 시 코드 수정이 필요하다. 향후 **config.yaml** 등으로 분리하면 유지보수성이 개선될 것으로 보인다.
 - 검증 임계값 근거 보강: 기존 범위(-50~60도) 등 일부 검증 기준은 임의로 설정한 값으로, 실제 도메인 스펙(기상청 자료 등)을 참조해 임계값의 근거를 보강할 필요가 있다.
-